COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 009B - Object Oriented Programming 2	**SECTION:**	CPE21S3
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/15/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort.

Code + Console Screenshot	

```cpp
// DISPLAY ARRAY
template <typename T>
void display(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```cpp
int main() {

    // Array with 100 elements
    const int SIZE = 100;
    int unsortedArr[SIZE];

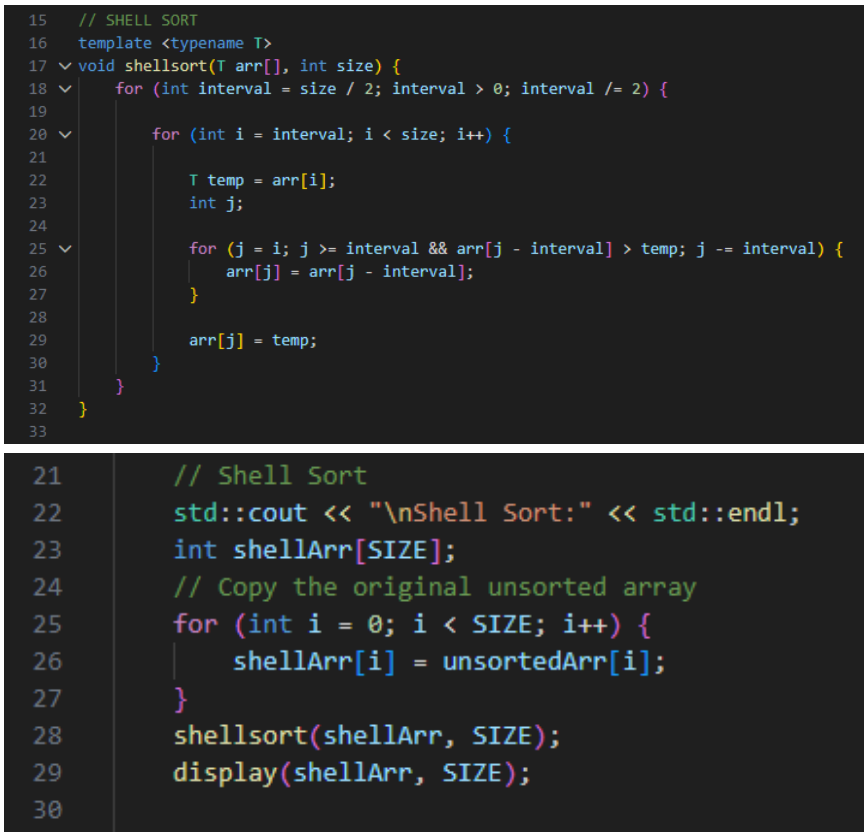
    // Generate random values
    srand(time(0));
    for (int i = 0; i < SIZE; i++) {
        unsortedArr[i] = rand() % 100 + 1;
    }
    // Unsorted Array
    std::cout << "Unsorted Array:" << std::endl;
    display(unsortedArr, SIZE);

```

Observation	In the **main()** function, SIZE is set to 100, and an integer array named unsortedArr is created with 100 elements. The **srand(time(0))** statement initializes the random number generator using the current time, allowing different random values to be generated each time the program runs. Another for loop fills the array with random numbers using **rand() % 100 + 1**, which produces values

	from 1 to 100. The program prints and calls **display(unsortedArr, SIZE)** to show all 100 generated values. Since the values are randomly generated, the array will generally have a different order.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observation	The **shellsort()** function is a template that allows it to work with different data types. It starts by setting the interval or gap to half of the array size and gradually reduces the gap by half until it becomes zero. For each gap, the program compares elements that are separated by the specified interval and shifts larger elements to the right until each value is placed in its correct position. The temp variable is used to temporarily store the current element during the sorting process. In the **main()** function, the original unsortedArr is copied into a new array called shellArr so that the original data remains unchanged. The **shellsort()** function is then called to sort shellArr, and the **display()** function prints the sorted values.

```
// SHELL SORT
template <typename T>
void shellsort(T arr[], int size) {
    for (int interval = size / 2; interval > 0; interval /= 2) {

        for (int i = interval; i < size; i++) {

            T temp = arr[i];
            int j;

            for (j = i; j >= interval && arr[j - interval] > temp; j -= interval) {
                arr[j] = arr[j - interval];
            }

            arr[j] = temp;
        }
    }
}
```

```
    // Shell Sort
    std::cout << "\nShell Sort:" << std::endl;
    int shellArr[SIZE];
    // Copy the original unsorted array
    for (int i = 0; i < SIZE; i++) {
        shellArr[i] = unsortedArr[i];
    }
    shellsort(shellArr, SIZE);
    display(shellArr, SIZE);
```

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
// MERGE SORT
template <typename T>
void merge(T arr[], int left, int middle, int right) {

    int n1 = middle - left + 1;
    int n2 = right - middle;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[middle + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }

        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
```

```cpp
    // Merge Sort
    std::cout << "\nMerge Sort:" << std::endl;
    int mergeArr[SIZE];
    // Copy the original unsorted array
    for (int i = 0; i < SIZE; i++) {
        mergeArr[i] = unsortedArr[i];
    }
    mergesort(mergeArr, 0, SIZE - 1);
    display(mergeArr, SIZE);
```

```cpp
template <typename T>
void mergesort(T arr[], int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;
        // Sort first half
        mergesort(arr, left, middle);
        // Sort second half
        mergesort(arr, middle + 1, right);
        // Merge both halves
        merge(arr, left, middle, right);
    }
}
```

Observation	The **mergesort()** function uses a divide-and-conquer approach by repeatedly dividing the array into two smaller halves until each section contains only one element. The **merge()** function then combines these smaller sections by comparing their elements and placing them in the correct order in the original array. Temporary arrays, L and R, are used to store the left and right halves during the merging process. In the **main()** function, the original unsortedArr is copied into mergeArr so that the original array remains unchanged, and **mergesort(mergeArr, 0, SIZE - 1)** is called to sort the 100 values.**The display()** function shows the sorted array.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
96   // QUICK SORT
97   template <typename T>
98   int partition(T arr[], int low, int high) {
99       T pivot = arr[high];
100      int i = low - 1;
101
102      for (int j = low; j < high; j++) {
103          if (arr[j] <= pivot) {
104              i++;
105              std::swap(arr[i], arr[j]);
106          }
107      }
108      std::swap(arr[i + 1], arr[high]);
109
110      return i + 1;
111  }
112
113  template <typename T>
114  void quicksort(T arr[], int low, int high) {
115      if (low < high) {
116          int pivot = partition(arr, low, high);
117          // Sort elements before pivot
118          quicksort(arr, low, pivot - 1);
119          // Sort elements after pivot
120          quicksort(arr, pivot + 1, high);
121      }
122  }
123
124  #endif
125
```

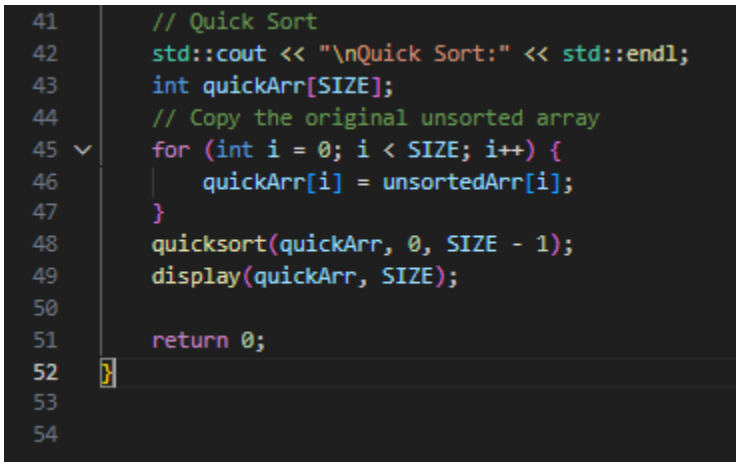
	
Observation	The Quick Sort first selects the last element of the array as the pivot. The **partition()** function then compares each element with the pivot and moves elements that are smaller than or equal to the pivot to the left side, while larger elements remain on the right side. After the partitioning is completed, the pivot is placed in its correct position in the array. The **quicksort()** function then uses recursion to sort the elements on the left side and the right side of the pivot separately. This process continues until each part contains only one element or no elements, meaning the array is already sorted. In the main.cpp, a separate quickArr array is created and the values from the original unsorted array are copied into it. The **quicksort()** function is then applied to quickArr, and the **display()** function prints the final sorted array

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, we can sort the left and right sublists from the partition method in Quick Sort using other sorting algorithms. For example, after choosing a pivot, the elements are divided into a left sublist and a right sublist. We can use Insertion Sort to sort both sublists.

Example:
Original array:
{8, 4, 7, 3, 9, 2, 6}

If 6 is selected as the pivot:
Left sublist: {4, 3, 2}
Pivot: 6
Right sublist: {8, 7, 9}

Apply Insertion Sort:

Left: {4, 3, 2} → {2, 3, 4}
Right: {8, 7, 9} → {7, 8, 9}

Final sorted array:
{2, 3, 4, 6, 7, 8, 9}
Quick Sort's partition can be combined with another sorting algorithm such as Insertion Sort to sort the sublists.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- In the given array **{4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}** For example, Quick Sort can have O(N log N) time performance for this array, when the pivots create very balanced splitting. Quick Sort is very quick, in general, as the sort sorts by partitioning the array into smaller arrays, sorting the smaller arrays, then merging. Merge Sort can have time complexity of O(N log N), as the array has log N levels of splitting (dividing the array in smaller pieces) and all elements are sorted at every level during the merging process. Quick Sort can also have time complexity of O(N log N) when the pivot partitions are balanced, as it too will have log N levels of splitting and sorting all array elements, but Quick Sort can degrade to O(N^2) worst case when the pivot is very small or very large dividing the array.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Ai Overview, w33school, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation of the concept for each type of sorting algorithm**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.